Homework 2 (CS59300 ASE)

Test Generation and Improvement Using Coding Agents

vichytong November 2, 2026

I used Codex to generate and improve automated tests for Papertrail, the research-paper web application from Homework 1. The experiment consisted of initial generation followed by two separately guided improvement rounds. The final suite contains 55 test methods: 52 pass, while three methods expose three known application defects through four failure events. Line coverage increased from 90.80% to 98.47%, and branch coverage increased from 77.59% to 91.95%.

This report was prepared with Codex assistance from the preserved English conversation, test outputs, coverage data, and experiment notes. The application was kept unchanged throughout the comparison. The test results and code diagnoses below are the agent's recorded work; they are not presented as my independent manual testing.

1. Test Generation Approach

Papertrail supports arXiv discovery, a persistent paper library, PDF uploads, LLM summaries, and paper-specific questions. The backend uses Flask, SQLite, requests, and pypdf; the frontend uses HTML, CSS, and JavaScript. I asked Codex to inspect the requirements, application code, and existing tests, then generate and evaluate an initial suite before making improvements.

My initial English request included:

> Generate meaningful, executable tests for the main functionality, including normal cases, edge cases, and error handling. Prefer the existing unittest framework unless there is a clear reason to change it.

I also required temporary storage, mocked external services, unchanged application code, coverage measurements, and preserved initial evidence. Codex used `unittest` and the Flask test client. The initial 37 methods were organized as follows.

Type	Methods	Main targets
Workflow integration	15	Search/save, persistence, uploads, deduplication, metadata editing, summaries, Q&A, and deletion.
Validation and errors	14	Invalid requests/PDFs, missing data or model setup, provider failures, authentication, and origin checks.
Components	8	Transactions, context selection, citations, provider requests, and cache reuse.

The fixtures contained known synthetic metadata, PDF text, and model responses. Assertions checked expected API results, SQLite records, stored files, provider arguments, and call counts. Only external boundaries were mocked where applicable; extraction, application routes, and storage ran. Dotenv loading was disabled, credentials were dummy values, and library data was temporary. The recorded runners blocked live HTTP/socket connections and recorded zero attempted connections.

The workspace already contained 14 Homework 1 methods and older Homework 2 artifacts. Codex inspected the older initial suite and helper as existing-test context, but preserved and excluded those older experiments from the reported measurements. The main comparison uses only the newly generated HW2 suite and its two improvements, in `initial_stage`, `round1_stage`, and `round2_stage`. [1–4]

2. Initial Test Evaluation

Code coverage and execution

The initial HW2-only run executed 37 methods: 35 passed and two failed, with three failing subcases, zero test errors, and zero skips. It ran on November 2, 2026, from 19:46:38 to 19:46:39 in America/Chicago and returned exit code 1 because it detected application defects.

All three stages measure the same four Python modules with coverage.py 7.17.0, `branch=True`, `config_file=False`, the same explicit include list, no omit list, and tracing before application import. The denominator is 522 executable statements and 174 branch destinations. Templates, frontend JavaScript/CSS, tests, dependencies, and submission tools are outside this coverage scope.

Module	Executed lines	Covered branch destinations
`app.py`	241/247 (97.57%)	68/74 (91.89%)
`database.py`	34/34 (100.00%)	2/2 (100.00%)
`pdf_tools.py`	83/111 (74.77%)	31/52 (59.62%)
`providers.py`	116/130 (89.23%)	34/46 (73.91%)
Overall	474/522 (90.80%)	135/174 (77.59%)

The main omissions were PDF size/page/text limits, fallback metadata extraction without document metadata, provider cache expiry and request spacing, and several redirect/download failures. Browser interaction and actual model answer quality were not evaluated by these backend percentages.

Test correctness and weak assertions

The recorded requirements/source review identified no incorrect initial expected value. Strong assertions checked exact persisted metadata and analysis history, extracted text, downloaded bytes, cache reuse, and unchanged state after rejected requests. Passing alone was not treated as proof of correctness.

Several checks were still weak. The search-timeout test began with an empty library, so accidental deletion could leave it empty and go unnoticed. A quota-retry test checked that one history entry existed without checking the exact saved answer. Some provider-error cases checked narrow status/message or file flags. The initial PDF fixtures also consistently supplied document metadata, leaving fallback extraction poorly exercised. These limitations motivated the improvement rounds. [2]

B1. Structured analysis kinds cause a server error

For a saved paper, `POST /api/papers/<id>/analyze` with `{"kind": []}` or `{"kind": {}}` returned HTTP 500 instead of the expected validation 400. Set membership was performed before type validation at `app.py:254`, causing an unhashable list/dict exception. The two inputs share one root cause.

The detecting method in `test_validation.py` is:

```
test_invalid_analysis_payloads_are_client_errors_without_provider_calls
```

The raw failure includes `AssertionError: 500 != 400`. The expected 400 follows the application's established invalid-input contract; the README does not enumerate these exact malformed values.

B2. Unicode pagination passes one validation but fails conversion

`GET /api/search?q=testing&page=%C2%B2` supplies the superscript character `page=` ². It passes `isdigit()` but cannot be parsed by `int()` at `app.py:156`. The endpoint returned HTTP 500 instead of a validation 400, without calling the search provider.

The detecting method is:

```
test_invalid_search_parameters_are_client_errors
```

An independent combined HW1+HW2 run reproduced B1 and B2. Its higher coverage is not used as the initial HW2 baseline. Full assertion traces and application diagnostics are preserved with the initial run. [2]

3. Iterative Test Improvement

Round 1: additional PDF requirements and representative fixtures

My first feedback focused on the initial PDF gaps. I asked for real limit boundaries, absence of records or orphaned files after rejection, metadata extraction without explicit document metadata, and graceful fallback when optional AI metadata enrichment fails. The English feedback included:

> For rejected uploads, verify that no paper record or orphaned PDF file is created.
>
> Explain how the expected values follow from the fixture and intended behavior. Flag ambiguous requirements rather than treating the current implementation as the specification.

Codex retained all 37 initial methods unchanged and added eight methods. The tests exercised readable PDFs at 20 MiB and one byte over, a separate multipart transport-cap rejection, 1/150 accepted pages and 0/151 rejected pages, and cumulative extracted-text totals of 999,999, 1,000,000, and 1,000,001 ASCII characters. They used real extraction and configured limits, without reducing thresholds. Rejected uploads were checked against both SQLite and the complete uploads directory.

Other fixtures removed the PDF Info dictionary so that visible title, author byline, and publication evidence had to be used. Missing author/year values were checked for honest unknown values. Four AI metadata failure cases—timeout, malformed JSON, wrong JSON shape, and invalid fields—checked that PDF-derived metadata, files, and a review notice survived, including after reopening the application.

The round executed 45 methods: 42 passed and three failed. Seven of the eight additions passed; all 35 previously passing methods still passed. Line coverage increased to 505/522 (96.74%) and branch coverage to 151/174 (86.78%). PDF-module coverage rose to 98.20% lines and 88.46% branches. [3]

B3. A continuation of the title was stored as an author

The new failing test used a PDF with two 24-point title lines above a separate 12-point byline:

```
Title:    Reliable Storage for
          Research Libraries
Byline:   Dana Tester and Evan Researcher
Expected: ['Dana Tester', 'Evan Researcher']
Actual:   ['Research Libraries', 'Dana Tester', 'Evan Researcher']
```

The detecting method in `test_pdf_round1.py` is:

```
test_multiline_prominent_title_is_not_misidentified_as_an_author
```

The title itself was joined correctly, but the earlier author scan treated the second title line as an author-like string. The extra author was present in both API output and SQLite. This is an accuracy defect for a clearly separated synthetic layout, not a claim that best-effort extraction must handle every multiline PDF.

The `20 MB` decimal/binary wording and the priority among conflicting publication dates remained ambiguous. The tests explicitly used the configured 20 MiB convention and clear publication evidence; they did not silently establish new date policies. B1 and B2 remained ordinary failures, giving four failure events across three defects. [3]

Round 2: coverage feedback and stronger assertions

For the second round, I directed Codex to the actual round 1 coverage JSON and evaluation notes. The provider module still missed 11 statements and 11 branch destinations, with 76.09% branch coverage. My feedback prioritized cache expiry/key separation, minimum request intervals, trusted redirects, bounded redirect failure, streamed-download limits, and interrupted downloads. It included:

> Test cache reuse before expiry and refresh at or after the documented ten-minute expiry.
>
> Also review existing provider and error-handling assertions. Strengthen any that could pass despite incorrect results or state, and explain concrete examples.

Codex added ten methods while retaining all 45 round 1 method identities. Three new methods checked cache expiry boundaries, distinct query/page/sort keys, and request spacing with controlled monotonic time. A virtual request at 1001 after one at 1000 required exactly 2.1 seconds of mocked waiting; no real sleeping was needed.

Three download methods checked permitted redirect destinations and exact bytes, the real streamed size boundary, and failure/retry behavior. Interrupted, oversized, excessively redirected, or HTTP-failed downloads had to preserve saved-paper state and leave no partial file. Four additional methods checked malformed/error-bearing feeds and recovery, unrelated feed entries, punctuation-only queries, and an unconfigured LLM client.

Four retained methods received stronger assertions. Redirect rejection now checks the actual destination and disabled automatic redirection. The timeout test seeds an existing paper and checks that it survives. The download-failure test compares the complete record, content/analysis state, and all upload entries. Quota retry verifies the exact saved answer, task/model/evidence, and subsequent cache reuse. These changes protect behavior that coverage percentages alone cannot distinguish.

The final run executed 55 methods: 52 passed and three failed, with the same four failure events as round 1. All ten additions and four strengthened methods passed. No new defect, incorrect expected outcome, or setup problem was identified. Provider coverage reached 98.46% lines and 95.65% branches. HTTP and the provider's time reference were controlled; actual cache decisions, parsing, streaming, application routes, extraction, and storage ran as applicable. [4]

4. Final Evaluation

The following comparison uses only the generated HW2 tests, with identical application hashes, coverage settings, and denominators. Earlier measurements were preserved and reused; the initial and round 1 suites were not rerun to manufacture the final comparison.

Metric	Initial	Round 1	Round 2
Methods	37	45	55
Passed / failed methods	35 / 2	42 / 3	52 / 3
Failure events	3	4	4
Test errors / skips	0 / 0	0 / 0	0 / 0
Executed lines	474/522	505/522	514/522
Line coverage	90.80%	96.74%	98.47%
Covered branch destinations	135/174	151/174	160/174
Branch coverage	77.59%	86.78%	91.95%
Distinct observed defects	2	3	3

Module (line / branch)	Initial	Round 1	Round 2
`app.py`	97.57 / 91.89	98.38 / 91.89	98.38 / 91.89
`database.py`	100 / 100	100 / 100	100 / 100
`pdf_tools.py`	74.77 / 59.62	98.20 / 88.46	98.20 / 88.46
`providers.py`	89.23 / 73.91	91.54 / 76.09	98.46 / 95.65

All entries in this table are percentages.

Round 1 added 31 executed statements and 16 branch destinations, mostly in PDF processing, and exposed B3. Its lower pass rate reflected a newly detected defect, not regression of previously passing tests. Round 2 added nine statements and nine branch destinations, all in the provider module, while strengthening checks of already covered state and results. It found no additional application defect, so greater fault-detection capability beyond B1–B3 was not empirically demonstrated.

Compared with the initial suite, the final suite gained 40 statements and 25 branch destinations: 7.66 and 14.37 percentage points, calculated from raw counts before rounding. Application and database coverage did not improve in round 2, even though several API state assertions became stronger. None of these changes repaired the application or removed known-defect tests.

Line coverage is executed statements divided by 522; branch coverage is covered branch destinations divided by 174. The text report's `Cover` column combines both metrics: the final value is $674/696 = 96.84\%$, rounded to 97%, rather than the 98.47% line metric. Failing tests also contribute coverage. These distinctions prevent a high execution percentage from being mistaken for correctness.

The recorded review identified no incorrect expected values, but it is an agent review of source, documented behavior, and synthetic fixtures, not independent certification. Remaining gaps include cache eviction and optional model request settings, concurrent changes, complex PDF layouts, disk failures, real network behavior, browser interaction, and real LLM quality. No mutation testing was performed. [2–4]

5. Reflection

Codex understood the main backend workflows well enough to generate tests that connected API responses, database state, stored PDFs, and provider calls. It correctly treated extracted PDF text as a prerequisite for analysis and recognized persistence and cache reuse as observable behavior. However, its initial choice of PDFs with explicit document metadata concealed fallback extraction. Broad coverage did not mean that the input space was representative.

The initial suite was strongest at normal integration flows, malformed inputs, isolated fixtures, mocked failures, and concrete request/result checks. It was weaker when the test needed to represent an existing populated state, distinguish the correct saved result from a count, or express a temporal property. An immediate-repeat cache test can pass even when expiry or key separation is wrong.

The two feedback strategies helped in different ways. Additional requirements changed the PDF fixtures and boundary cases, which found a new defect. Coverage and execution feedback then directed attention to missed provider paths, while assertion review improved tests that already executed the relevant code. This showed why *generate more tests* would be less informative than naming a behavior, supplying representative evidence, and asking what a test would reject.

The experiment also illustrates the limits of mocks. Matching a mocked answer to a saved answer verifies transport and storage, not whether the answer is scientifically correct. Synthetic PDFs cannot establish behavior for arbitrary columns, equations, scripts, or conflicting dates. Database coverage of 100% does not establish crash recovery or concurrent correctness. One recorded run per improvement round cannot establish statistical freedom from flakiness.

My main lesson from these results is to treat generated tests as reviewable claims about intended behavior. Coverage helps prioritize inspection; correctness still depends on the expected result and the assertion. I would keep a fixed application snapshot, preserve the initial result before improvement, make ambiguities explicit, and review data/state checks alongside status codes.

I directed the staged English requests; Codex generated the tests, fixtures, runners, assertion changes, notes, and this report. A separate Chinese planning conversation helped prepare the English feedback. No significant manual test changes, application repairs, or corrected expected outcomes are recorded. The reflection here is based on the observed experiment rather than an additional personal manual-testing session.

6. GitHub Repository

Repository: `https://github.com/VichyTong/CS593-ASE`.

The application is in `solutions/hw1/`. The final test suite is in `solutions/hw2/round2_stage/tests/`, with five test modules: `test_components.py`, `test_validation.py`, `test_workflows.py`, `test_pdf_round1.py`, and `test_providers_round2.py`. Each stage also contains its own runner, dependency file, English feedback, design notes, and preserved evidence.

Installing dependencies and running the final suite

Run from the repository root using Python 3.11 or newer. The recorded environment used Python 3.11.4, Flask 3.1.3, pypdf 6.19.0, requests 2.34.2, python-dotenv 1.2.4, gunicorn 26.2.0, and coverage 7.17.0. These fresh-environment commands are reproduction instructions, not additional recorded experiment runs.

```
python3 -m venv /tmp/papertrail-hw2-venv
TASK_PY=/tmp/papertrail-hw2-venv/bin/python
"$TASK_PY" -m pip install -r \
  solutions/hw2/round2_stage/requirements-test.txt
"$TASK_PY" -B solutions/hw2/round2_stage/run_round2.py \
  --output /tmp/papertrail-hw2-round2-run
```

Choose a new output directory for every run. The runner writes `results.json`, test output, diagnostics, native coverage data, JSON/text coverage, and HTML coverage. It verifies the frozen inputs and protected inventories. This requires the complete preserved `initial_stage` and `round1_stage` directories, including normally ignored evidence, plus every protected historical file listed in the round 2 manifest.

On the unchanged snapshot the expected result is 55 methods, 52 passed, three failed methods, four failure events, zero test errors, and zero skips. The runner returns exit code 1 because B1–B3 remain present. B1 accounts for two failing subcases, B2 for one, and B3 for one ordinary failure. The tests are runnable; a passing suite is not claimed.

Code-only coverage reproduction

A source-only checkout may lack historical integrity artifacts. With the application, final tests, and dependency files present, the following POSIX-shell recipe executes the final suite without historical-inventory checks while measuring the same four modules. It is an alternative recipe, not a fourth experimental stage. After the expected nonzero test exit, run the coverage-report commands.

```
TASK_SCOPE='solutions/hw1/app.py,solutions/hw1/database.py'
TASK_SCOPE="$TASK_SCOPE,solutions/hw1/pdf_tools.py"
TASK_SCOPE="$TASK_SCOPE,solutions/hw1/providers.py"
PYTHONPATH=solutions/hw1 "$TASK_PY" -B -m coverage run \
  --rcfile=/dev/null --branch --include="$TASK_SCOPE" \
  --data-file=/tmp/papertrail-hw2.coverage \
  -m unittest discover -s solutions/hw2/round2_stage/tests -v
"$TASK_PY" -B -m coverage json --rcfile=/dev/null \
  --data-file=/tmp/papertrail-hw2.coverage \
  -o /tmp/papertrail-hw2-coverage.json
"$TASK_PY" -B -m coverage html --rcfile=/dev/null \
  --data-file=/tmp/papertrail-hw2.coverage \
  -d /tmp/papertrail-hw2-coverage-html
```

Evidence and references

All paths below are relative to the repository root. The displayed English prompt passages are exact excerpts with line wrapping adjusted, not fabricated conversation screenshots.

[1] `solutions/hw1/README.md` and the four application modules: intended features and source-level explanations.

[2] `solutions/hw2/initial_stage/`: `generation-request.txt`, `interaction-excerpts.txt`, `experiment-notes.txt`, `evidence/initial-only/`, and `evidence/integrity-check.json`.

[3] `solutions/hw2/round1_stage/`: `feedback-request.txt`, `design-and-setup.txt`, `experiment-notes.txt`, `evidence/round1-only/`, and `evidence/integrity-check.json`.

[4] `solutions/hw2/round2_stage/`: `feedback-request.txt`, `design-and-setup.txt`, `experiment-notes.txt`, `evidence/round2-only/`, `evidence/comparison.json`, `evidence/retained-test-changes.diff`, and `evidence/integrity-check.json`.

Within each recorded run directory, `results.json` contains counts/events, `coverage.json` contains raw metrics, `test-output.txt` contains assertions, and `diagnostics.txt` contains application traces. Each stage also preserves pre-execution frozen inputs and a final evidence archive.

The runs took place on November 2, 2026: initial 19:46:38–19:46:39, round 1 20:39:41–20:39:55, and round 2 22:07:59–22:08:14, all America/Chicago. The initial stage also executed legacy-only and combined suites, excluded from the main comparison. Each improved suite ran once. Integrity records confirm unchanged application code and earlier stages. Report preparation did not rerun application tests. Remote publication of the GitHub test artifacts and grader access were not verified during report preparation.